

[2025_개인 프로젝트 결과보고서(201969022배성하)]

LSTM AutoEncoder를 통한 이상탐지 모델

- S H A P 과 C T G A N 의 활 용 -

1. 서론

가. 연구 배경

- 이상 금융거래 탐지시스템(FDS)의 중요성
- 지도학습 모델의 한계
- 금융 데이터의 불균형 문제
- AutoEncoder의 적용 가능성

나. 연구 목표

- LSTM AutoEncoder와 SHAP_VALUE를 통한 이상탐지모델

다. 연구 방법

- 데이터 레이블링 문제 해결
- 데이터 불균형 해소
- 컴퓨팅 자원 효율화

라. 데이터 및 전처리 과정

- 사용한 데이터셋 설명 (Kaggle Credit Card Fraud)
- 전처리 방식
- 이상 데이터 비율 조정 방식
- SHAP Value를 활용한 변수 선택
- 상관관계 및 로지스틱 회귀 분석

2. 모델 구성 및 실험 설계

가. LSTM AutoEncoder 기본 구조 및 SHAP 적용

나. 다양한 레이어 수 및 구조별 실험

다. 하이퍼파라미터 설정

- LSTM Units
- 레이어 수
- Dropout, Learning Rate, Activation, Batch Size

라. 모델 학습 결과

- 상위 성능 조합
- 하위 성능 조합

3. 연구 결과 분석

가. 전체 평균 성능 지표 요약 (표/그래프 포함)

- Column Selection 적용X, dropout적용
- Window Sliding 방식, SHAP 및 dropout적용
- Non_overlapping방식, SHAP 및 dropout적용
- logitstic regression에 의한 변수 선택

나. 연구 결과

4. 결론 및 시사점

1. 서론

가. 연구 배경

1) 이상 금융거래 탐지시스템(FDS)의 중요성

□ 핀테크 기술의 발달과 함께 금융 거래의 편리함이 증가하며 현금 거래는 감소하는 반면 신용 카드를 통한 거래는 지속적으로 증가함. 이로 인해 이상 금융거래 탐지시스템(FDS)의 중요성이 대두되고 있음.

2) 지도학습 모델의 한계

□ 실시간 거래 금융 데이터는 레이블링 되어 있지 않음. 지도학습 모델은 학습을 위해 정상과 이상의 레이블이 충분히 갖춰져야 하는 어려움이 존재함.

3) 금융데이터의 불균형 문제

□ 신용카드 거래 데이터와 같은 금융 데이터는 이상 거래의 비율이 정상 거래의 비율보다 현저히 적게 나타나는 심각한 데이터 불균형 문제를 지니고 있음.

4) AutoEncoder의 적용 가능성

□ 금융 거래에서는 정상 거래의 데이터 수집에 비해 이상 거래의 데이터 수집은 훨씬 어려움. AutoEncoder는 지도학습모델에 비해 정밀도나 재현율 측면에서 한계가 있을 수 있으나, 의료분야나 금융분야처럼 이상데이터를 수집하기 의료, 금융 등 이상 데이터를 확보하기 어려운 도메인에서는 오토인코더가 현실적이고 효과적인 대안이 될 수 있음.

나. 연구 목표

1) LSTM AutoEncoder와 SHAP_VALUE를 통한 이상탐지모델

□ 시계열 거래 데이터를 기반으로 기존의 지도학습 모델의 대안으로, 기존 논문에서 제시한 LSTM AutoEncoder를 활용한 이상 거래 탐지 모델을 재현하는 것이다. 탐지 결과의 성능을 위해 SHAP_Value를 활용하여 구현한다.

다. 연구 방법

1) 데이터 레이블링 문제 해결

- ☐ 데이터 레이블링 문제를 해결하기 위해 레이블이 없는 원본 데이터를 활용하여 모델이 스스로 정상 패턴을 학습하게 하는 방식인 자기지도학습 기반의 LSTM Autoencoder를 활용

2) 데이터 불균형 문제 해소

- ☐ 데이터 불균형 문제를 일부 해소하기 위해 정형 데이터에 특화된 Conditional tabular GAN (CTGAN)을 활용하여 이상 데이터의 비율을 오버샘플링

3) 컴퓨팅 자원 효율화

- ☐ 컴퓨팅 리소스를 절약하고 모델 성능을 개선하기 위해 설명 가능한 AI (XAI) 기법인 SHAP를 사용하여 주요 변수를 추출

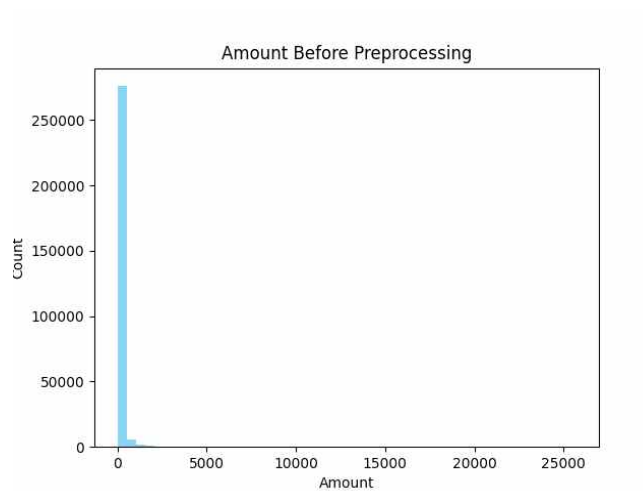
라. 데이터 및 전처리 과정

1) 사용한 데이터셋 설명 (Kaggle Credit Card Fraud Dataset)

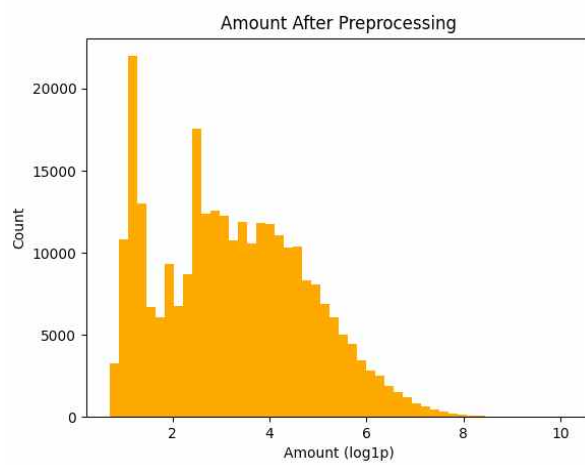
- ☐ 2013년 유럽에서 사용된 신용카드 거래 데이터로, 이틀 동안 발생한 거래내역을 사용
- ☐ 데이터 세트의 총 거래 건수는 약 28만 개이며, 이 중 사기 데이터는 492건으로 전체 데이터의 약 0.172%를 차지
- ☐ 데이터는 주요 변수인 V1부터 V28까지와 'Time', 'Amount', 'Classes'를 포함하여 총 31개 변수로 구성
- ☐ 데이터 세트의 주요 변수(V1, V2, ... V28)는 개인정보보호를 위해 주성분 분석(PCA)을 통해 비식별화(anonymized)가 되어 있음.

2) 전처리 방식

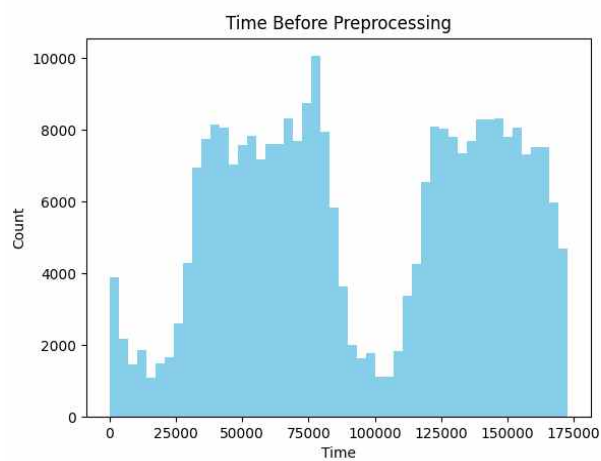
- ☐ 초 단위로 작성된 'Time' 변수는 일 중 시간으로 변환
- ☐ 비교적 편차가 큰 'Amount' 변수에는 로그 스케일(log scale)을 사용함.



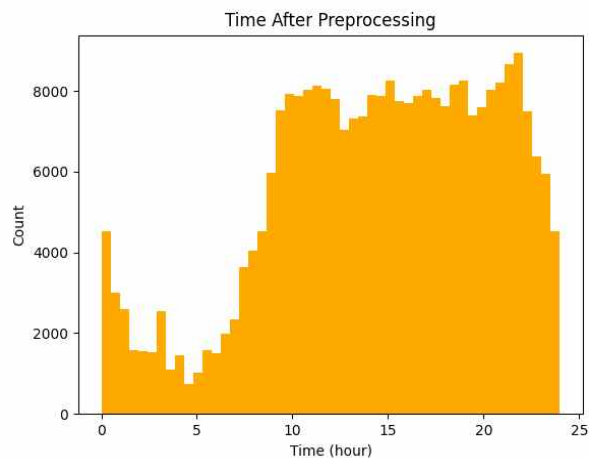
[Figure1] Amount Before Preprocessing



[Figure2] Amount After Preprocessing



[Figure3] Time Before Preprocessing



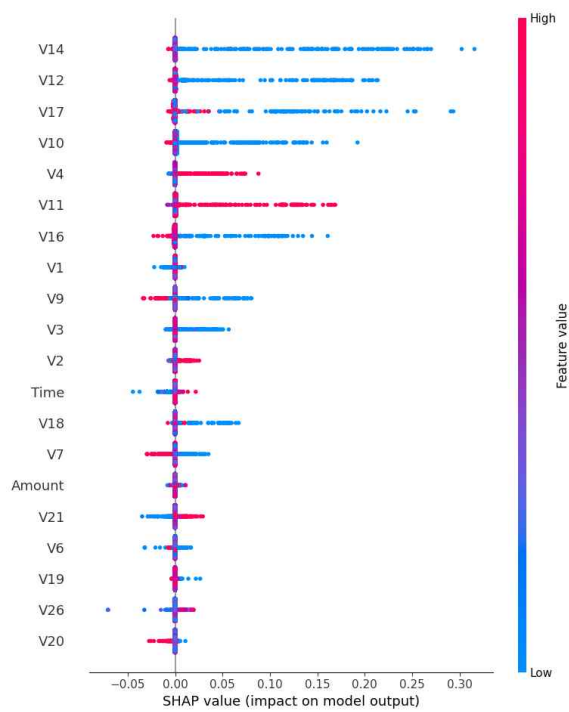
[Figure4] Time After Preprocessing

3) 이상 데이터 비율 조정 방식

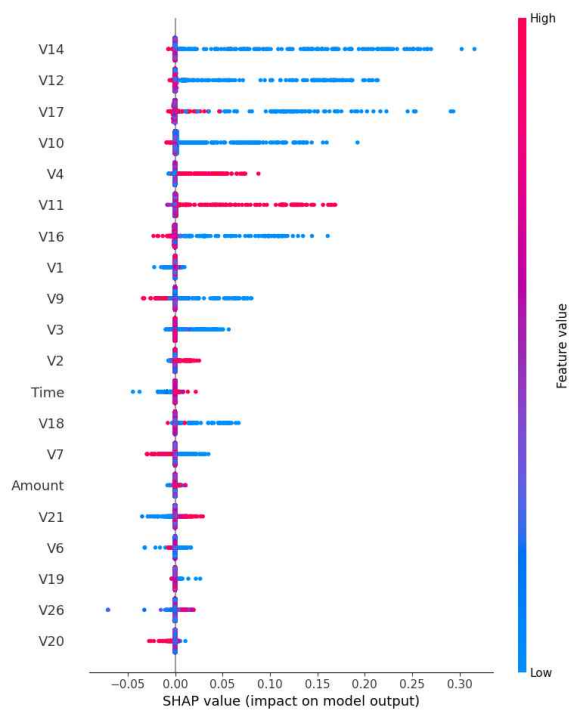
- ☐ 정형 데이터의 특성과 거래 데이터의 개별적 특성을 고려하여 Conditional Tabular GAN (CTGAN) 알고리즘을 활용하여 오버샘플링을 진행.

4) SHAP Value를 활용한 데이터 선택

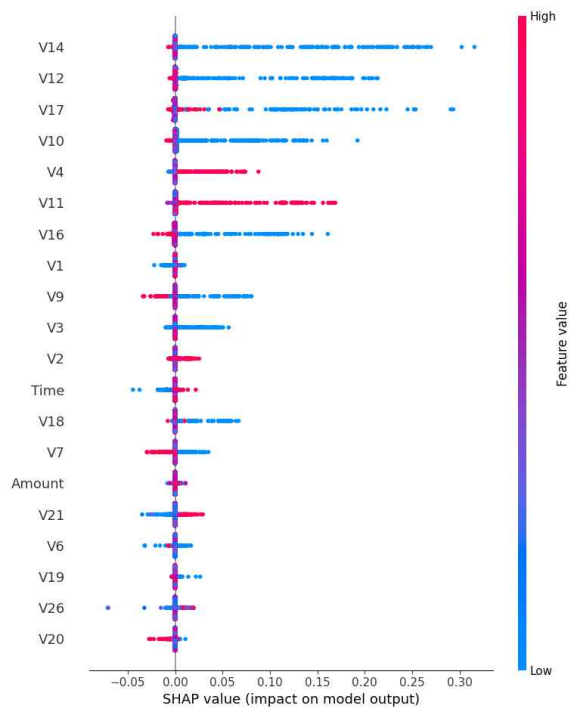
- ☐ RandomForest모델을 바탕으로 SHAP Value를 계산함.
- ☐ 기존 Raw Data 및 CTGAN을 활용하여 증강한 데이터에 Seed값을 0으로 동일하게 주어 SHAP value를 계산하였는데, CTGAN의 여부와 상관없이 SHAP Value가 동일하게 나옴.



[Figure5] Raw Data-Shap_Value



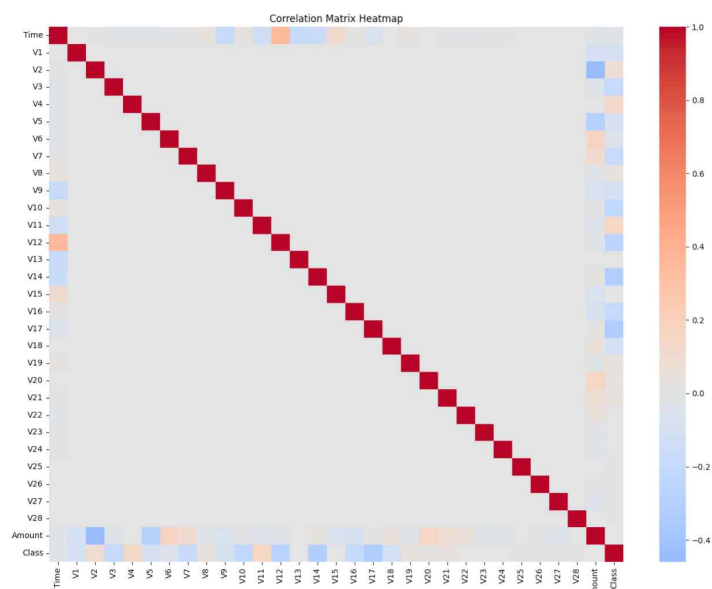
[Figure6] CTGAN 0.03%Data-Shap_Value



[Figure7] CTGAN 0.07%Data-Shap_Value

5) 상관관계 및 로지스틱 회귀 분석

- 이상값 레이블인 ‘Class’ 와 다른 특성 간의 상관관계가 SHAP Value와 비슷한 추이를 보여줌.



[Figure8]상관관계 분석 히트맵

- 로지스틱 회귀에서 회귀계수(coefficient)로 각 독립변수가 타겟값(Class)에 얼마나 영향을 미치는지 수치적으로 파악하고 이를 토대로 상위 변수 20개를 선택하여 비교군 실험을 진행함.

Current function value: 0.005150
Iterations 12

Logit Regression Results

Dep. Variable:	Class	No. Observations:	305715
Model:	Logit	Df Residuals:	305684
Method:	MLE	Df Model:	30
Date:	Fri, 30 May 2025	Pseudo R-squ.:	0.9797
Time:	20:03:53	Log-Likelihood:	-1574.3
converged:	True	LL-Null:	-77541.
Covariance Type:	nonrobust	LLR p-value:	0.000

	coef	std err	z	P> z	[0.025	0.975]
Intercept	-8.9784	0.237	-37.816	0.000	-9.435	-8.505
Amount	-0.1074	0.039	-2.774	0.006	-0.183	-0.032
Time	0.0746	0.011	6.863	0.000	0.053	0.096
V1	-0.0533	0.014	-3.724	0.000	-0.081	-0.025
V2	0.1066	0.017	6.354	0.000	0.074	0.139
V3	0.0156	0.017	0.917	0.359	-0.018	0.049
V4	0.7090	0.033	21.219	0.000	0.644	0.774
V5	-0.2202	0.019	-11.750	0.000	-0.257	-0.183
V6	-0.1915	0.033	-5.868	0.000	-0.255	-0.128
V7	-0.2019	0.017	-12.173	0.000	-0.234	-0.169
V8	-0.0366	0.012	-2.947	0.003	-0.061	-0.012
V9	-0.0082	0.041	-0.199	0.843	-0.089	0.073
V10	-0.3737	0.033	-11.492	0.000	-0.437	-0.310
V11	0.1849	0.036	5.096	0.000	0.114	0.256
V12	-0.3175	0.029	-11.002	0.000	-0.374	-0.261
V13	0.1553	0.057	2.739	0.006	0.044	0.266
V14	-0.4002	0.026	-15.153	0.000	-0.452	-0.348
V15	-0.3077	0.058	-5.302	0.000	-0.421	-0.194
V16	-0.0927	0.031	-2.956	0.003	-0.154	-0.031
V17	-0.0934	0.017	-5.395	0.000	-0.127	-0.059
V18	-0.3239	0.038	-8.621	0.000	-0.398	-0.250
V19	-0.3129	0.049	-6.428	0.000	-0.408	-0.217
V20	-0.1277	0.030	-4.279	0.000	-0.186	-0.069
V21	0.0394	0.015	2.661	0.008	0.010	0.068
V22	-0.1156	0.032	-3.645	0.000	-0.178	-0.053
V23	-0.0884	0.021	-4.308	0.000	-0.129	-0.048
V24	-0.5547	0.102	-5.414	0.000	-0.756	-0.354
V25	-0.4487	0.077	-5.864	0.000	-0.599	-0.299
V26	0.3620	0.123	2.932	0.003	0.120	0.604
V27	-0.3431	0.056	-6.109	0.000	-0.453	-0.233
V28	-0.1370	0.066	-2.077	0.038	-0.266	-0.008

[Figure9]로지스틱 회귀 분석

2. 모델 구성 및 실험 설계

가. LSTM AutoEncoder 기본 구조 및 SHAP 적용

- LSTM Autoencoder 모델을 활용함. LSTM Autoencoder의 기본적인 구조는 LSTM 구성 요소로 이루어진 인코더(encoder)와 디코더(decoder)로 구성됨. 이 모델은 정상 데이터의 특징을 학습하도록 훈련
- 임의의 SHAP_VALUE를 바탕으로 특성을 절반으로 줄여 모델 학습과 예측 및 평가를 진행함.
- AutoEncoder의 특정 하이퍼파라미터가 일관되게 좋은 결과를

보장하지 않음을 알 수 있었음.

- ☐ 본 실험에서는 Leaky_Lelu함수를 활용하여 Dropout은 0.1로 적용하였으며 은닉층은 4개의 레이어(20-16-12-8)로 진행하고 Adam을 활용함.

나. LSTM AutoEncoder 다양한 레이어 수별 구조

레이어 수	인코더 구조	디코더 구조
2	8 → 4	4 → 8
2	8 → 6	6 → 8
2	12 → 4	4 → 12
2	12 → 6	6 → 12
2	16 → 4	4 → 16
2	16 → 6	6 → 16
3	8 → 6 → 4	4 → 6 → 8
3	8 → 7 → 6	6 → 7 → 8
3	12 → 8 → 4	4 → 8 → 12
3	12 → 9 → 6	6 → 9 → 12
3	16 → 10 → 4	4 → 10 → 16
3	16 → 11 → 6	6 → 11 → 16
4	8 → 6 → 5 → 4	4 → 5 → 6 → 8
4	12 → 9 → 6 → 4	4 → 6 → 9 → 12
4	12 → 10 → 8 → 6	6 → 8 → 10 → 12
4	16 → 12 → 8 → 4	4 → 8 → 12 → 16
4	16 → 12 → 9 → 6	6 → 9 → 12 → 16
5	8 → 7 → 6 → 5 → 4	4 → 5 → 6 → 7 → 8
5	12 → 10 → 8 → 6 → 4	4 → 6 → 8 → 10 → 12
5	12 → 10 → 9 → 7 → 6	6 → 7 → 9 → 10 → 12
5	16 → 13 → 10 → 7 → 4	4 → 7 → 10 → 13 → 16
5	16 → 13 → 11 → 8 → 6	6 → 8 → 11 → 13 → 16

다. 하이퍼파라미터 구성

1) 실험에 사용된 하이퍼파라미터 목록

- ☐ LSTM Units (최대) [8, 12, 16]
- ☐ LSTM Units (최소) [4, 6]
- ☐ 레이어 수 [2, 3, 4, 5]
- ☐ Dropout Rates: [0.1, 0.15, 0.2]
- ☐ Learning Rates: [0.0005, 0.001, 0.005, 0.01]
- ☐ Activations: ['leaky_relu', 'relu', 'tanh']
- ☐ Batch Size: 32, 64
- ☐ optimizer: Adam

라. 모델 학습 결과 상위 10개 항목

units_nax	units_min	layers	dropout	learnin g_rate	batch _size	activ ation	precision	recall	f1
12	6	3	0.2	0.005	32	relu	0.649485	0.331579	0.439024
12	6	2	0.2	0.001	32	tanh	0.67033	0.321053	0.434164
12	6	2	0.2	0.001	64	tanh	0.67033	0.321053	0.434164
12	4	4	0.15	0.0005	32	relu	0.67033	0.321053	0.434164
8	4	2	0.1	0.0005	32	leaky_r elu	0.645833	0.326316	0.433566
8	4	2	0.1	0.0005	32	tanh	0.645833	0.326316	0.433566
8	4	2	0.1	0.0005	64	leaky_r elu	0.645833	0.326316	0.433566
8	4	2	0.1	0.0005	64	relu	0.645833	0.326316	0.433566
8	4	2	0.1	0.0005	64	tanh	0.645833	0.326316	0.433566

마. 모델 학습 결과 하위 10개 항목

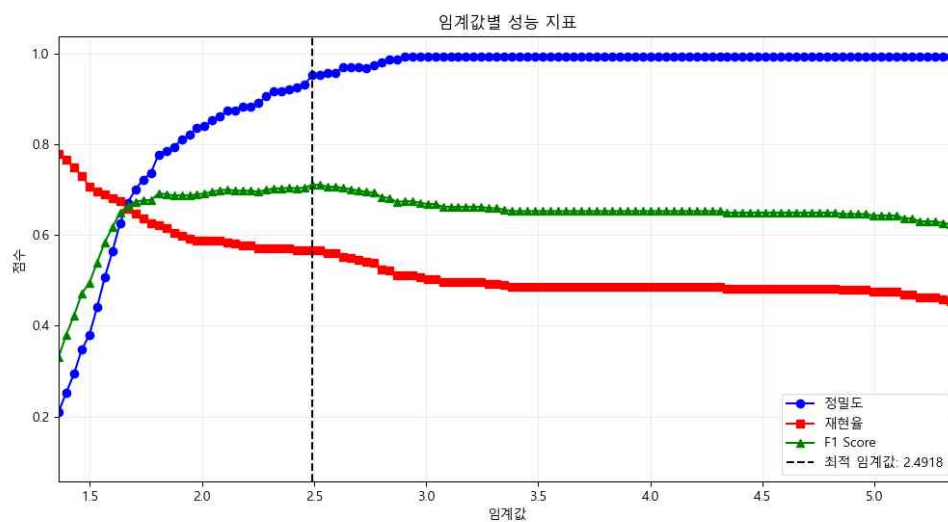
units_nax	units_min	layers	dropout	learnin g_rate	batch _size	activ ation	precision	recall	f1
16	4	3	0.1	0.001	32	tanh	0.632653061	0.326315789	0.430555556
16	4	3	0.1	0.005	64	tanh	0.632653061	0.326315789	0.430555556
16	6	3	0.15	0.005	64	relu	0.632653061	0.326315789	0.430555556
8	4	4	0.1	0.0005	32	tanh	0.632653061	0.326315789	0.430555556
8	4	4	0.15	0.0005	32	tanh	0.632653061	0.326315789	0.430555556
16	4	4	0.1	0.001	64	relu	0.632653061	0.326315789	0.430555556
12	4	5	0.1	0.0005	32	leaky_r elu	0.632653061	0.326315789	0.430555556
12	6	5	0.2	0.01	64	relu	0.632653061	0.326315789	0.430555556
16	6	5	0.2	0.01	64	leaky_r elu	0.632653061	0.326315789	0.430555556

3. 연구 결과 분석

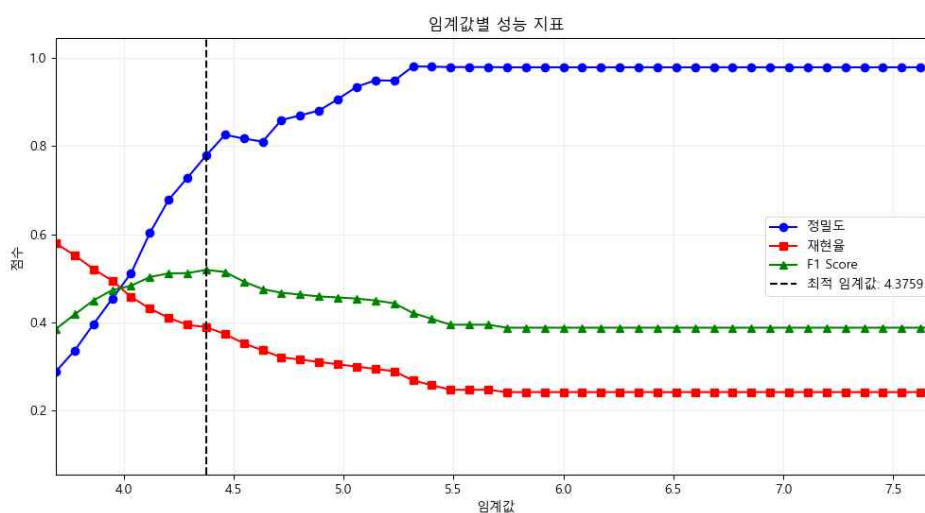
가. 전체 평균 성능 지표 요약 (표/그래프 포함)

□ Column Selection 적용X, dropout적용

	Precision	Recall	F1_Score
timesteps10	0.9529	0.5664	0.7105
timesteps30	0.7789	0.3895	0.5193
timesteps50	0.6585	0.3158	0.4269
timesteps70	0.4593	0.3899	0.4218



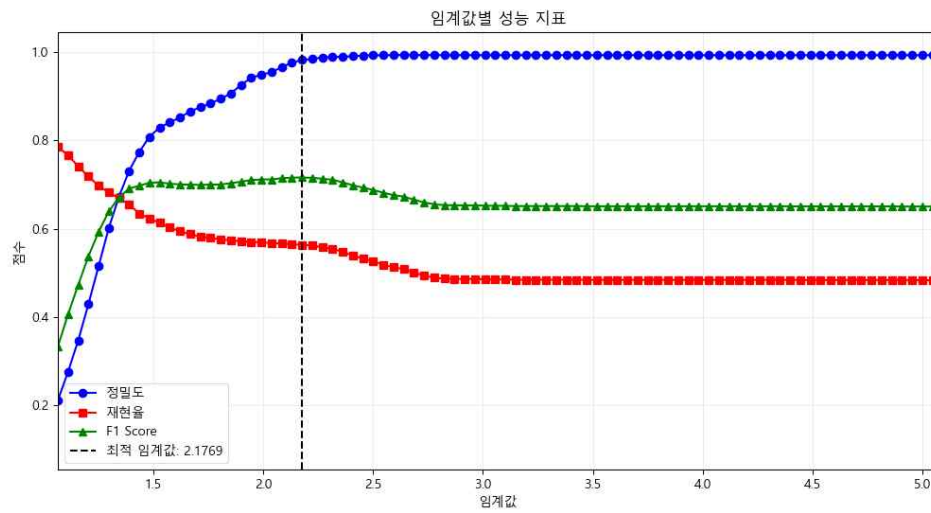
[Figure10] Time Steps 10, Dropout적용



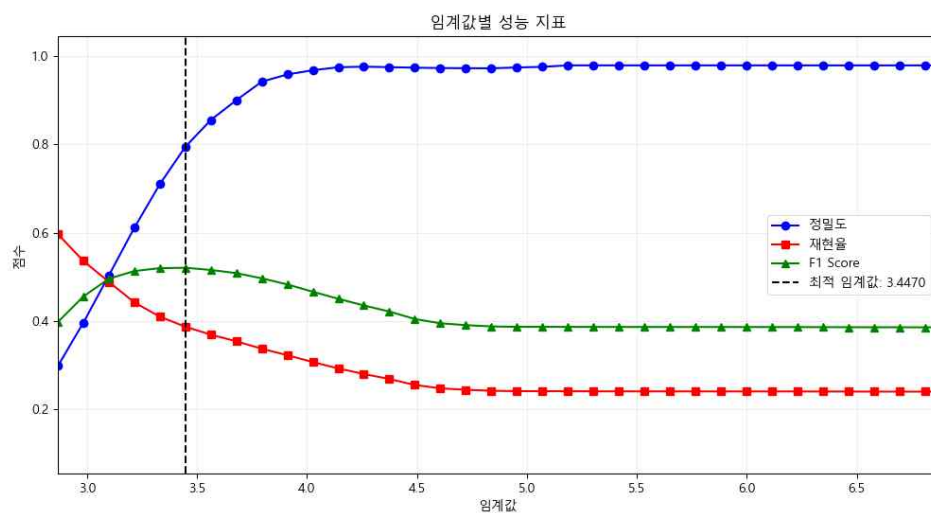
[Figure11] Time Steps 30, Dropout적용

□ Window Sliding 방식, Shap 및 dropout적용

	Precision	Recall	F1_Score
timesteps10	0.9818	0.5637	0.7162
timesteps30	0.7947	0.3873	0.5208
timesteps50	0.6052	0.3705	0.4597
timesteps70	0.5166	0.3845	0.4409



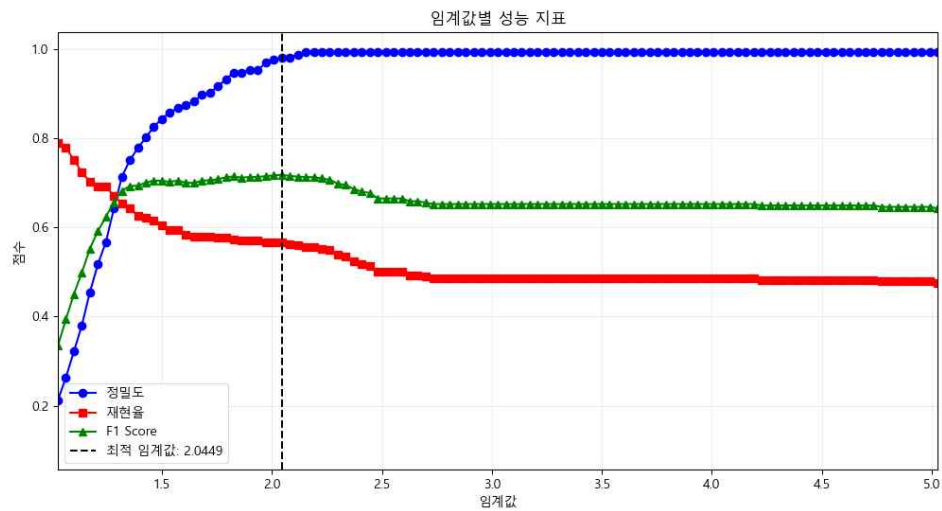
[Figure12] Time Steps 10, Window_Sliding방식, Shap 및 Dropout적용



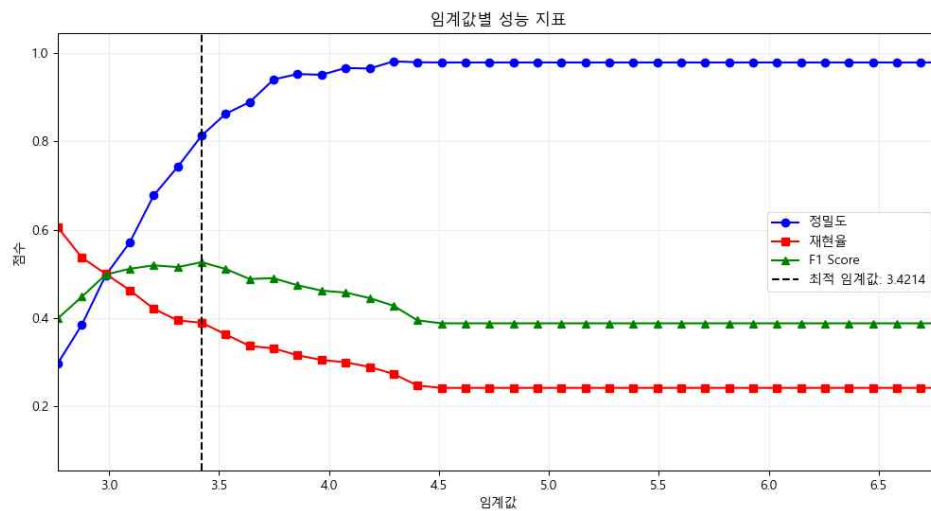
[Figure13] Time Steps 30, Window_Sliding방식, Shap 및 Dropout적용

□Non_overlapping방식, Shap 및 dropout적용

	Precision	Recall	F1_Score
timesteps10	0.9818	0.5664	0.7184
timesteps30	0.8132	0.3895	0.5267
timesteps50	0.6224	0.3567	0.4535
timesteps70	0.6023	0.3333	0.4291



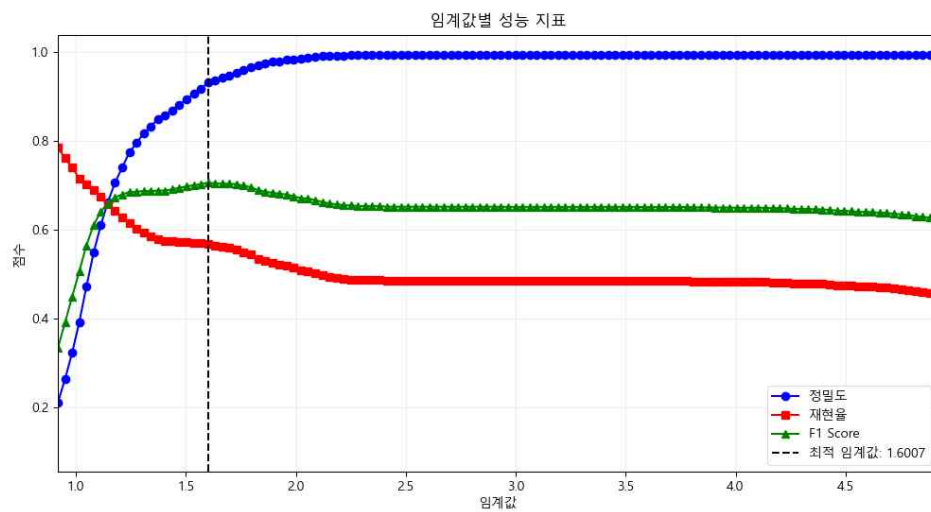
[Figure14] Time Steps 10, Non_overlapping방식, Shap 및 Dropout적용



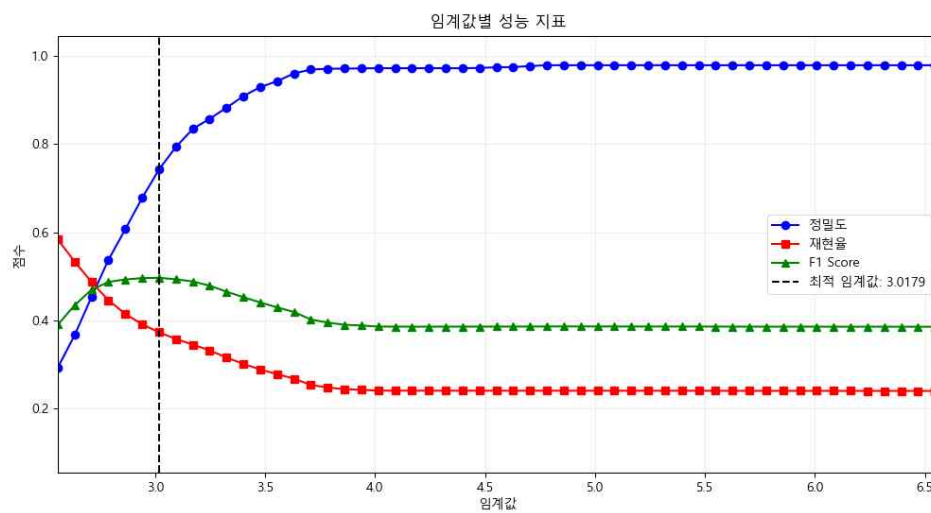
[Figure15] Time Steps 30, Non_overlapping방식, Shap 및 Dropout적용

□logistic regression에 의한 변수 선택

	Precision	Recall	F1_Score
timesteps10	0.9304	0.5682	0.7056
timesteps30	0.7427	0.3733	0.4968
timesteps50	0.5630	0.3615	0.4403
timesteps70	0.4797	0.3802	0.4242



[Figure16] Time Steps 10, WindowSliding, Dropout적용, 로지스틱회귀



[Figure17] Time Steps 30, WindowSliding, Dropout적용, 로지스틱회귀

나. 연구 결과

TimeSteps10일 때 SHAP을 적용하고 겹치는 WindowSliding방식을 적용하지 않고 겹치지 않는 시퀀스 생성함수를 사용했을 때 Dropout을 적용했을 때 0.7184로 미미하지만 가장 높은 f1_score를 기록했다. 다음으로 WindowSliding방식으로 겹치도록 시퀀스를 생성하여 진행했을 때 0.7162의 성능을 보였다. 이때 SHAP을 적용하지 않고 전체 특성을 사용했을 때 오히려 0.7105로 성능이 떨어졌는데, 이는 불필요한 변수가 포함될 경우 오히려 재구성 오류가 불안정해짐을 시사한다. 또한 로지스틱 회귀를 기반으로 변수를 선택하였을 때는 0.7056으로 가장 낮은 성능을 보였는데 이는 회귀계수를 통한 변수 선택은 중요한 변수를 누락시킬 수 있음을 시사한다. 짧은 시계열 구간에서는 SHAP변수기반 및 겹치지 않는 시퀀스 생성방식이 가장 효과적임을 알 수 있었다.

TimeSteps가 30으로 늘어날 경우 대부분 모델에서 F1_Score가 감소하는 경향이 나타났다. 이 구간에서도 TimeSteps10과 마찬가지로 SHAP+겹치지 않는 시퀀스 생성방식에서 0.5267로 가장 높은 성능을 보였으며 SHAP+겹치는 WindowSliding시퀀스 생성방식이 0.5208로 근소한 성능 차이를 보였다. 또한 SHAP을 사용하지 않고 회귀 기반 특성 선택 또한 가장 낮은 f1_score를 도출하였다.

TimeSteps50부터는 SHAP을 이용한 방법들의 상대적 우위가 낮아지는 모습을 보였는데 SHAP 및 WindowSliding방식을 적용했을 때 가장 높은 f1_score가 계산되었으며, 특이하게도 로지스틱회귀를 기반으로 변수를 선택한 모델이 TimeSteps10, 30일 때와 달리 전체 특성을 사용한 모델보다 더 나은 결과를 보였다.

TimeSteps70에서는 50때와 마찬가지로 WindowSliding+SHAP방식이 가장 높은 성능을 보여주었으며 전체 특성으로 AutoEncoder를 학습하는 것보다 로지스틱회귀를 통해 변수선택을 한 모델이 미미하지만 조금 더 높은 f1_score값을 도출하였다. 이를 통해 TimeSteps50이상부터는 불필요한 특성이나 정보가 오히려 잡음을 유발하며 SHAP이나 회귀 기반 변수 선택이 모델의 학습 안정성에 기여한다는 점을 시사한다.

4. 결론 및 시사점

오늘날 현금 결제 비율은 20% 미만으로 감소한 반면, 신용카드를 이용한 금융 서비스의 이용률은 지속적으로 증가하고 있다. 이러한 상황에서 점점 교묘해지는 금융사기 수법에 대응하기 위해, 본 연구에서는 이상 금융 거래 탐지시스템(Fraud Detection System, FDS)에 관한 프로젝트를 수행하였다. 금융권 도메인에서는 이상 거래(사기 거래) 데이터를 수집하기 어려운 특성이 있으므로, 본 연구에서는 정상 데이터만을 활용하여 학습하는 자기지도학습 기반의 알고리즘을 사용하여 이상 거래를 탐지하였다. 또한, 금융 거래 데이터에 대한 즉각적인 대응을 위해, SHAP Value를 활용하여 불필요한 특성을 제거함으로써 컴퓨팅 자원 소모를 줄이고자 하였다.

제안된 LSTM AutoEncoder 모델은 SHAP을 활용한 변수 선택과 Dropout 등의 정규화 기법을 적용했을 때 가장 높은 F1-score(0.7184)를 기록하였으며, 특히 짧은 시계열 구간(TimeSteps=10)에서는 비중첩 시퀀스 방식이 유의미한 성능 향상을 보였다. 반면, 전체 특성을 활용하거나 로지스틱 회귀 기반 변수 선택을 활용한 경우에는 성능이 오히려 감소하였는데, 이는 불필요한 정보가 재구성 오류를 증가시키고 모델 학습을 불안정하게 만든다는 점을 시사한다. 또한, TimeSteps가 길어질수록 SHAP의 상대적 우위는 감소하였고, 이 구간에서는 WindowSliding 방식과 회귀 기반 변수 선택 방식이 상대적으로 나은 성능을 보였다. 이를 통해, 시계열 길이에 따라 최적의 시퀀스 구성 및 변수 선택 전략이 달라져야 함을 확인할 수 있었다.

향후에는 SHAP_Value 계산 시 사용하는 모델의 최적화, 그리고 실시간 탐지 시스템으로의 확장을 염두해둘 수 있겠다. 또한 실제 금융 도메인 지식 기반의 변수 선택 전략이 통합될 시, 실제 금융 현장에서의 활용도를 높이는 데 기여할 수 있을 것으로 예상된다.